Analytics Vidhya

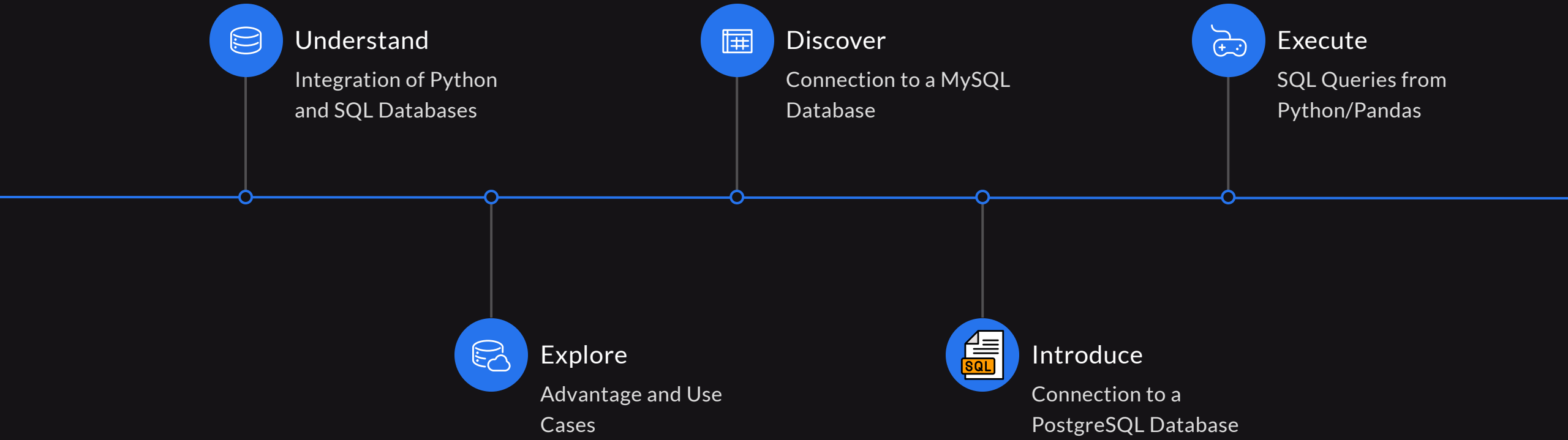
Connecting Python to SQL Databases

Enabling AI Agents with Database Integration

Instructor

Prashant Sahu

Manager (Data Science), Analytics Vidhya


Understand
Integration of Python and SQL Databases
Discover
Connection to a MySQL Database
Execute
SQL Queries from Python/Pandas
Explore
Advantage and Use Cases
SQL
Introduce
Connection to a PostgreSQL Database

Agenda

Why Connect Python to SQL Databases?

Connecting Python to SQL databases offers numerous advantages.

Data Storage and Retrieval

Efficiently store large datasets

Retrieve data as for processing

Scalability and Performance

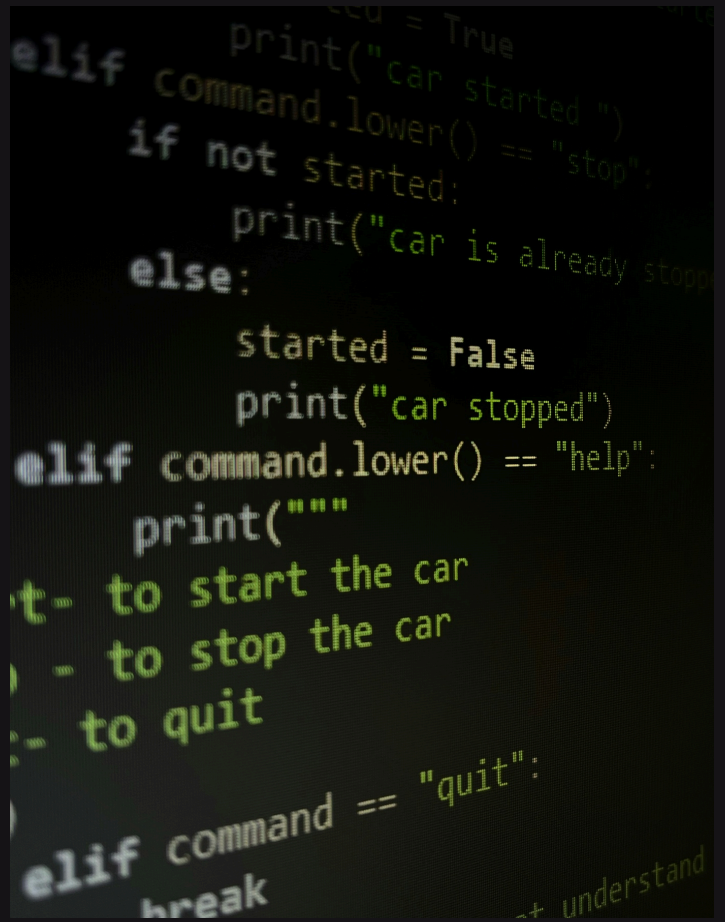
Handle increasing amounts of data smoothly

Real-Time Data Access

AI agents can access up-to-date information

Data Preprocessing

Clean and prepare data within Python for AI models



Analytics Vidhya

Why Connect Python to SQL Databases?

Connecting Python to SQL databases offers numerous use-cases.

AI Agents Needing Dynamic Data

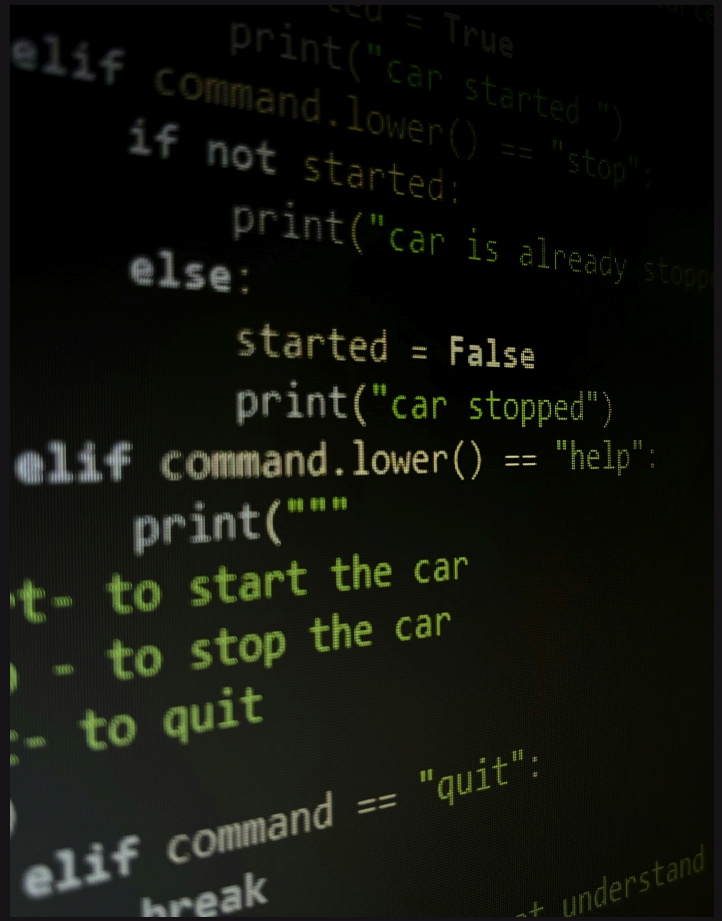
Chatbots accessing customer info

Generating AI Applications

Using stored datasets for training and inference

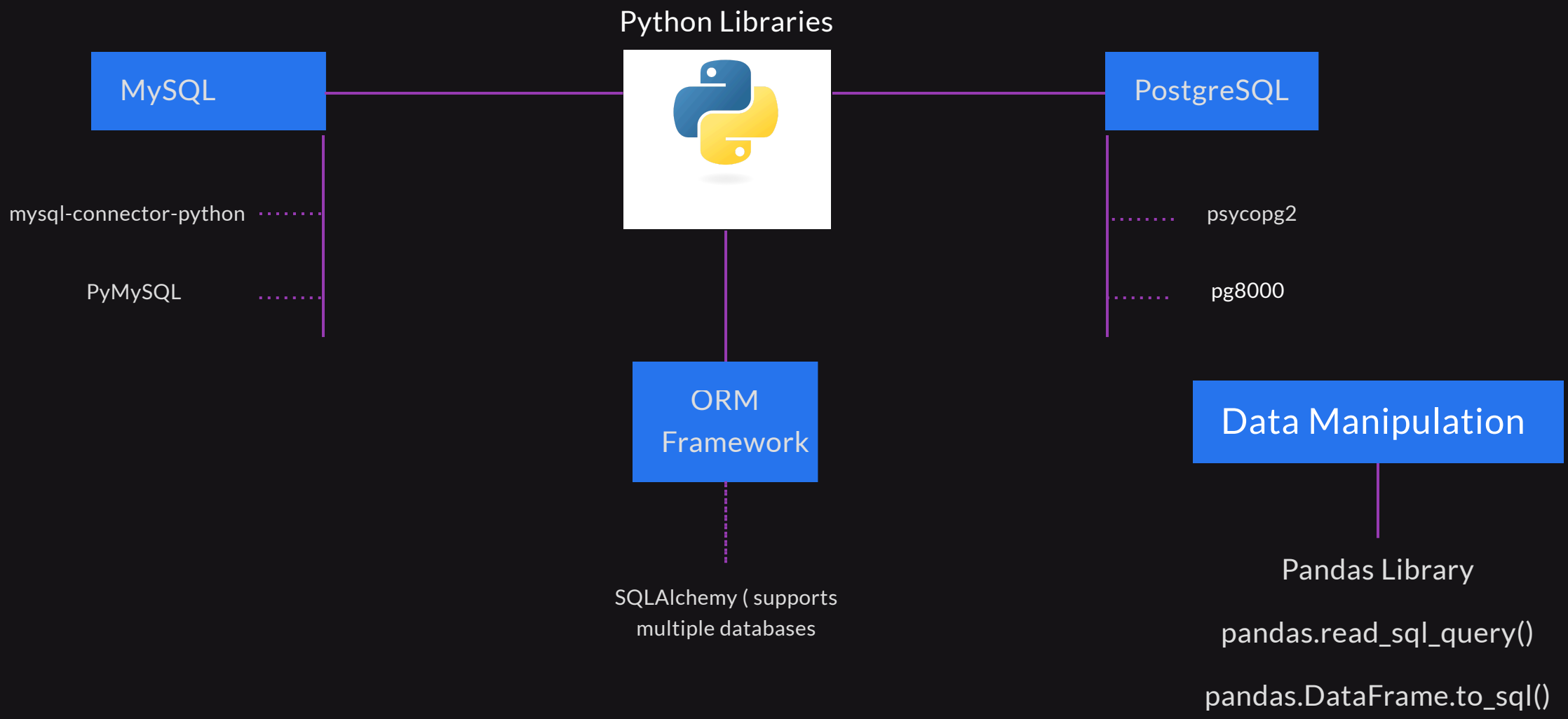
Logging and Monitoring

Store interactions logs for analysis and improvement



Analytics Vidhya

Tools for Connecting Python to SQL Databases



Analytics Vidhya

Connecting to a MySQL Database

```
## Code Example Using mysql-connector-python ##

# Import the MySQL connector library
import mysql.connector

# Establish a connection to the MySQL database
try:
    connection = mysql.connector.connect(
        host='localhost',         # Database host
        user='your_username',     # Your username
        password='your_password',# Your password
        database='your_database' # Database name
    )
    print("Connection successful!")
except mysql.connector.Error as err:
    print(f"Error: {err}")
```

Connecting to a PostgreSQL Database

```python
## Code Example Using psycopg2 ##

# Import the psycopg2 library
import psycopg2

# Establish a connection to the PostgreSQL database
try:
    connection = psycopg2.connect(
        host='localhost',         # Database host
        user='your_username',     # Your username
        password='your_password',# Your password
        database='your_database' # Database name
    )
    print("Connection successful!")
except psycopg2.Error as err:
    print(f"Error: {err}")
```

Executing SQL Queries from Python/Pandas

```
## Fetching Data into a Pandas DataFrame ##

# For MySQL using mysql-connector-python
import pandas as pd
import mysql.connector

# Establish the connection (as shown earlier)
connection = mysql.connector.connect(...)

# Write the SQL query
query = "SELECT * FROM your_table;"

# Execute the query and fetch data into a DataFrame
df = pd.read_sql_query(query, connection)

# Close the connection
connection.close()
```

Alternatives for Working with Databases

Limitations of direct SQL queries

Complexity

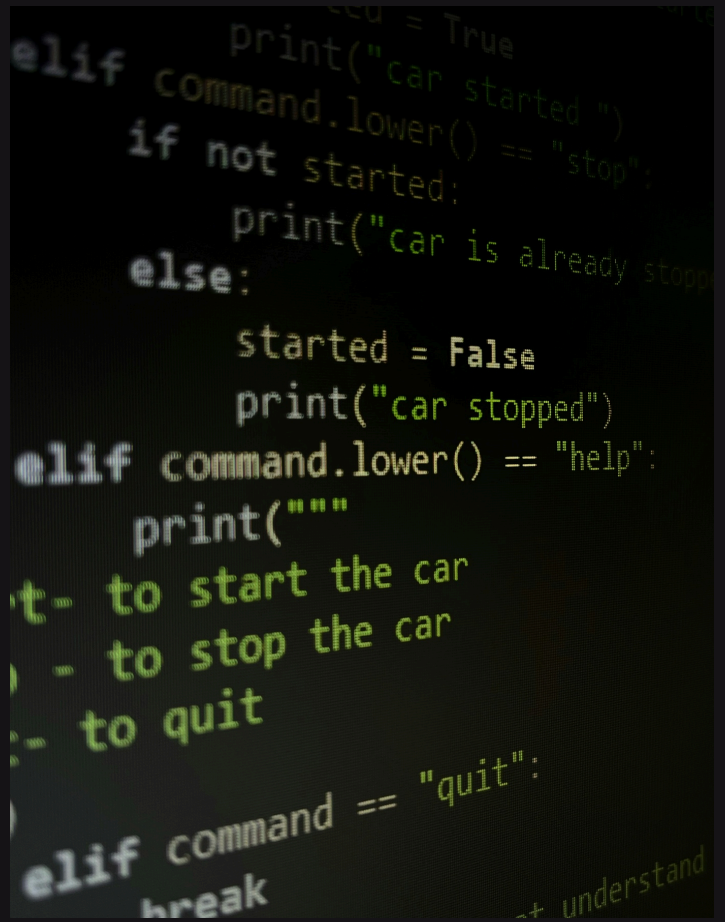
Writing raw SQL queries can be error-prone

Security Risks

Vulnerable to SQL injection if not handled properly

Maintainability

Harder to manage and refactor code



Analytics Vidhya

Alternatives for Working with Databases

Why use ORM framework?

Abstraction

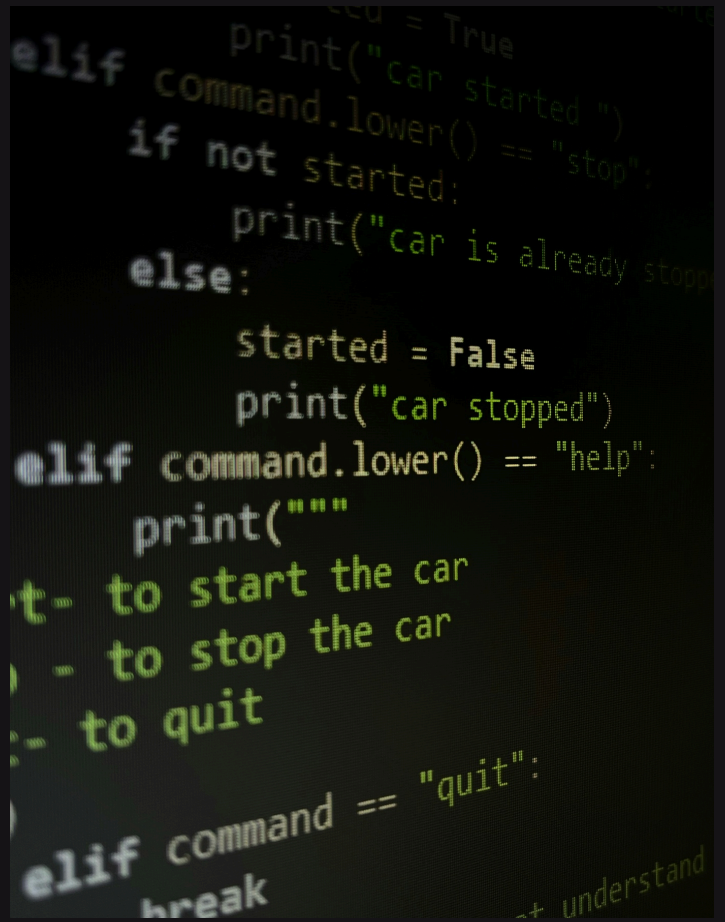
Interact with the database using Python classes and methods.

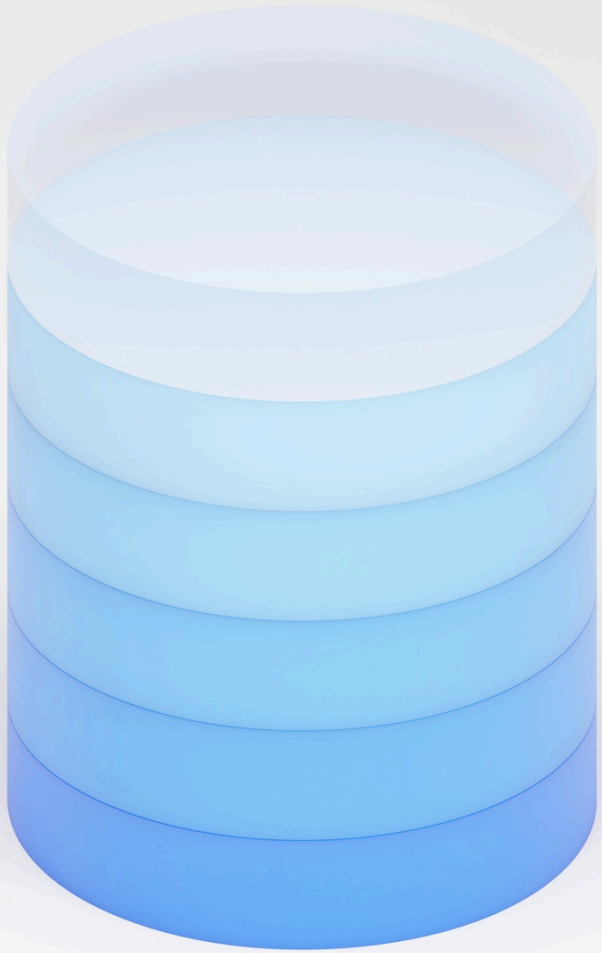
Productivity

Faster development with less boilerplate code.

Portability

Easier to switch between different database backends.





Introduction to SQLModel and SQLAchemy

SQLAlchemy

A powerful ORM for Python supporting multiple

Full control over SQL queries, support for advanced database features

SQLModel

A library built on top of SQLAlchemy and Pydantic.

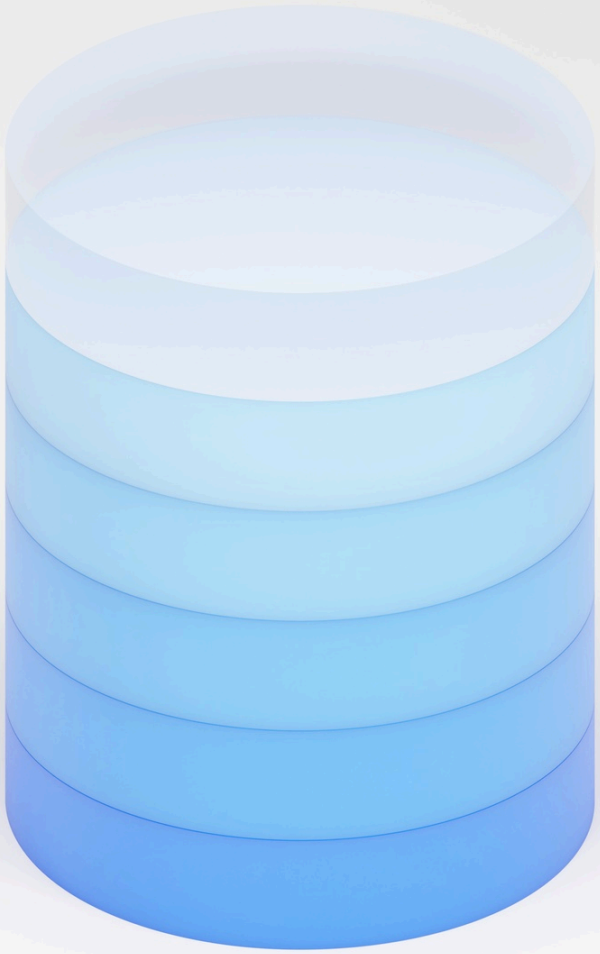
Integrates with FastAPI for building web applications

Benefits in AI Applications

Define and manage data schemas easily.

Automate data validation with Pydantic models.

Seamless integration with web framework like FastAPI.



Introduction to In-memory Databases with SQLite

Definition

A database that resides entirely in the system's RAM

Characteristics

Fast read/write operations due to memory access speeds

Data is volatile - lost when the applications terminates.

Creating an In-memory SQLite Database

```python
import sqlite3

# Create a connection to an in-memory SQLite database
connection = sqlite3.connect(':memory:')

# Create a cursor object to execute SQL commands
cursor = connection.cursor()

# Example: Create a table
cursor.execute('''
    CREATE TABLE users (
        id INTEGER PRIMARY KEY,
        name TEXT,
        email TEXT
    )
''')
```

```python
# Insert data into the table
cursor.execute('''
    INSERT INTO users (name, email)
    VALUES ('Alice', 'alice@example.com')
''')

# Commit changes
connection.commit()

# Query the data
cursor.execute('SELECT * FROM users')
result = cursor.fetchall()
print(result)  # Output: [(1, 'Alice', 'alice@example.com')]

# Close the connection
connection.close()
```

When to Use In-Memory Databases

Fast Data Access

Ideal for applications requiring quick read/write operations.

Testing and Development

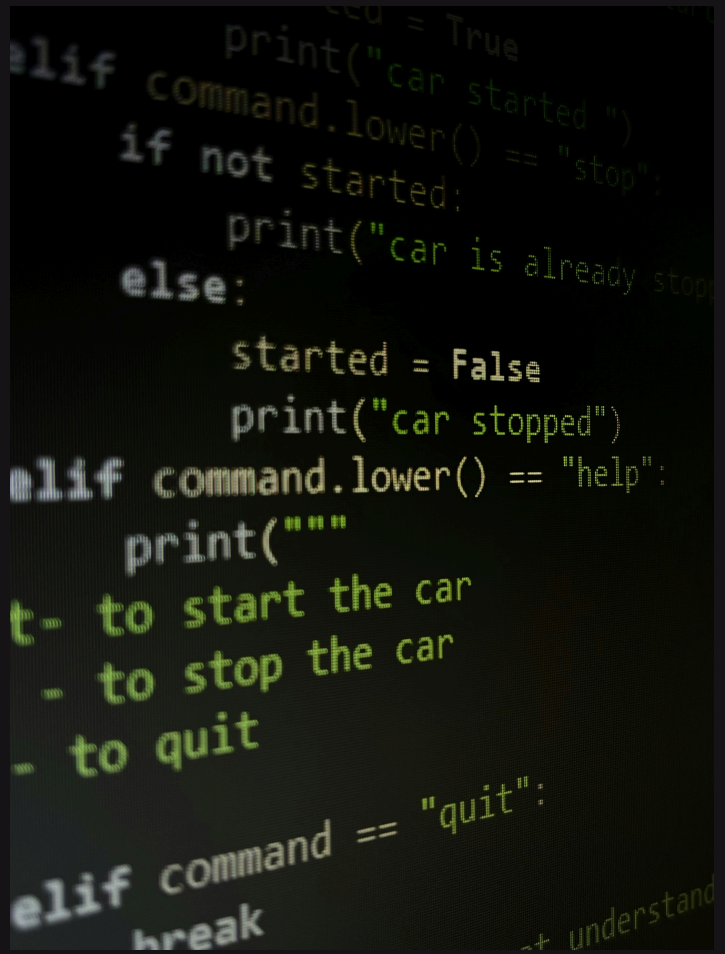
Simplifies testing by providing a fresh database state for each test run.

Temporary Data Storage

Suitable for caching or session data that doesn't need persistence.

Data Manipulation

Perform complex computations on data without affecting the production database.



Limitations of Using In-Memory Databases

Volatility

Data is lost when the connections is closed or the applications ends.

Memory Constraints

Limited by the amount of available RAM

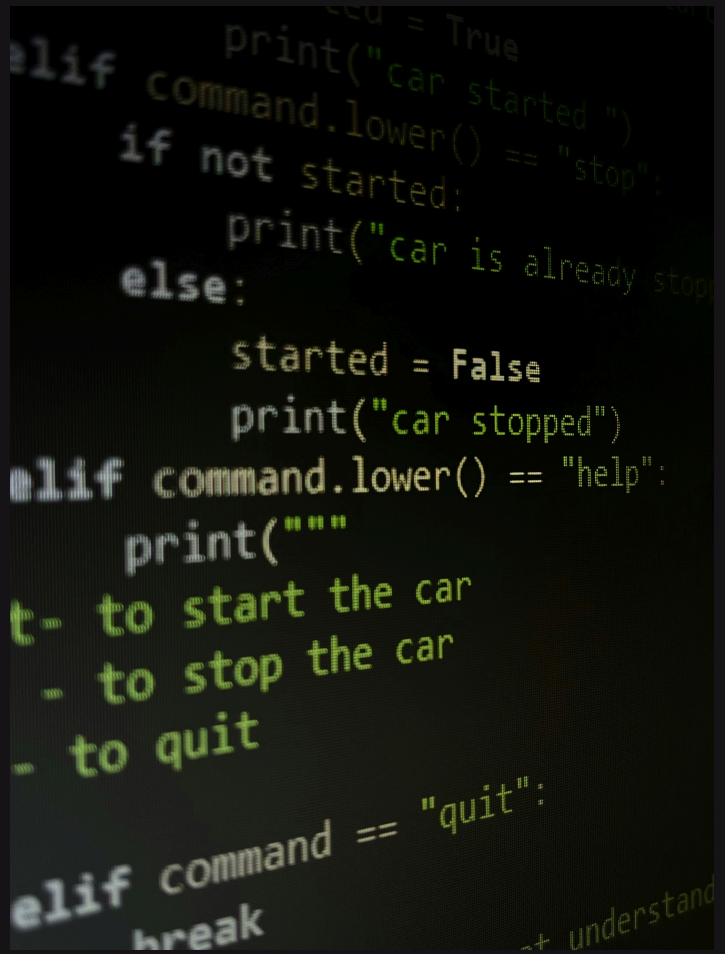
Not suitable for large datasets

Concurrency

May not handle multiple simultaneous connections well

Persistence

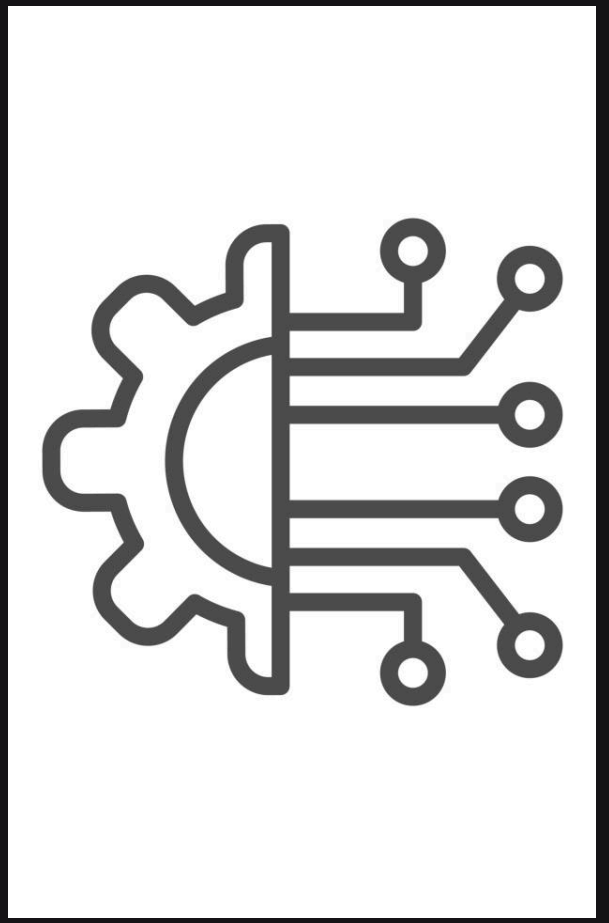
Not suitable for applications that require data to persist across sessions.



Summary

Key Takeaways

1. Connecting Python to SQL databases enables efficient data handling.
2. Use libraries like mysql-connector-python and psycopg2 for database connectivity.
3. Pandas integrates seamlessly with SQL databases for data manipulation.
4. ORM framework like SQLAlchemy and SQLModel offer advantages over raw SQL queries.



Analytics Vidhya